

Vector Pseudocode

Code	Cost	Times	Total
Read line	1	n	n
Split line	1	n	n
Create course	1	n	n
!Add to vector	1	n	n

Total Cost = $4n$

Runtime = $O(n)$

Hash Pseudocode

Code	Cost	Times	Total
Read line	1	n	n
Split line	1	n	n
Create course	1	n	n
Insert into hash table	1	n	n

Total Cost = $4n$

Runtime = $O(n)$

Tree Pseudocode

Code	Cost	Times	Total
Read line	1	n	n
Split line	1	n	n
Create course	1	n	n
Insert into tree	1	n	n^2

Total Cost = $n \log n$ (average), n^2 (worst case)

Runtime = $O(n \log n)$ on average, $O(n^2)$ in the worst case

Vector Pseudocode

A vector is simple and easy to work with, which makes it a good option for storing the course data. It's straightforward to add courses as they are read from the file, and the structure itself isn't complicated. However, it doesn't keep anything sorted, so the program would need an extra step to sort the data before printing it in order. Searching is also less efficient since it requires going through each course one by one, which gives it a linear runtime of $O(n)$.

Hash Pseudocode

A hash table is useful because it allows for fast searching. Once the course is stored using a hash function, it can be found quickly without having to go through everything. This makes it better than a vector when searching for a specific course. However, it doesn't store the courses in order, so printing them alphabetically isn't as easy. It also requires handling collisions, which adds a little more complexity.

Binary Search Tree

A binary search tree is helpful because it keeps the data sorted as it is inserted. This makes printing the courses in order much easier without needing an extra sorting step. Searching is also more efficient than a vector in most cases. One issue is that if the tree becomes unbalanced, it can slow things down and make it less efficient. Even with that, it still works well for this type of program.

Recommendation

Out of the three, I would choose the binary search tree. It keeps the courses in order automatically, which makes printing them easier and avoids needing an extra sorting step. It also provides faster searching than a vector, with an average runtime of $O(\log n)$. Even though the worst-case runtime can be $O(n^2)$ if the tree becomes unbalanced, it still offers the best balance between efficiency and functionality for this program.